

澳門大學
UNIVERSIDADE DE MACAU
UNIVERSITY OF MACAU



信息學院
Faculdade de Ciência da Informação e Computação
Faculty of Information Science and Computing

Classification

EMEN 7029 & EMEN 7104 - Special Topic In Electromechanical Engineering / Robotics

Qingbiao Li

University of Macau

Review 2
Classification 18
Probability Theory 21
Classification Model 35
Classification Loss 46
Classification Optimization 58
Coding 60

Review

Review

Last time, we reviewed derivatives

$$f'(x) = \frac{d}{dx}f = \frac{df}{dx} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

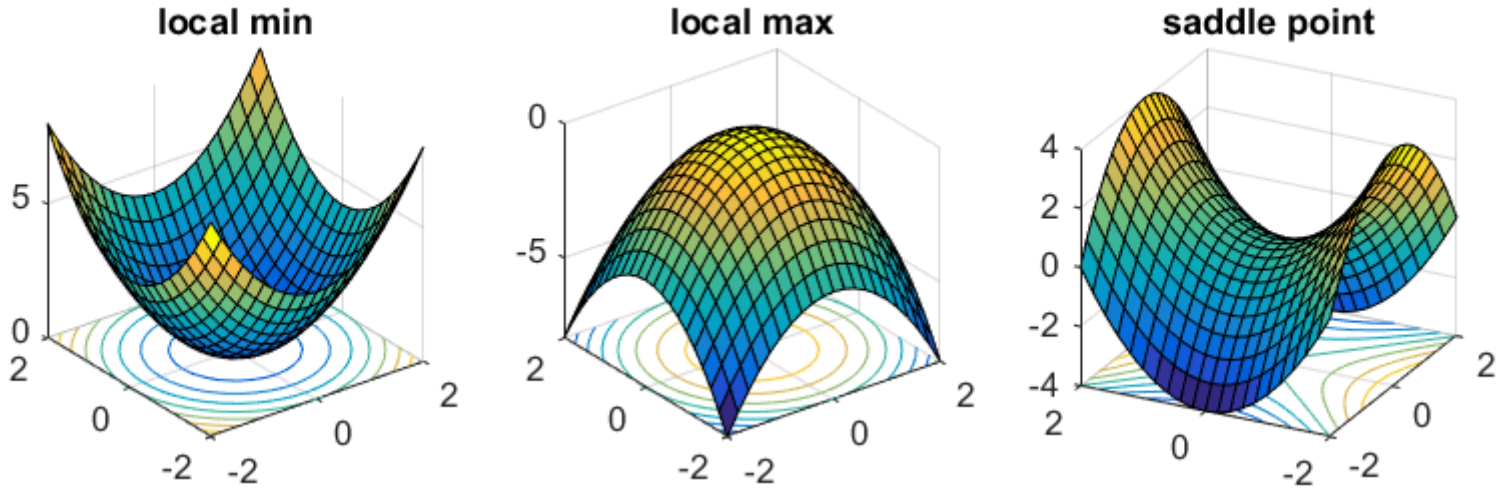
and gradients

$$(\nabla_{\boldsymbol{x}} f)(\boldsymbol{x}) = \begin{bmatrix} \frac{\partial f(\boldsymbol{x})}{\partial x_1} \\ \frac{\partial f(\boldsymbol{x})}{\partial x_2} \\ \vdots \\ \frac{\partial f(\boldsymbol{x})}{\partial x_n} \end{bmatrix}$$

Review

Gradients are important in deep learning for two reasons:

Reason 1: $f(\boldsymbol{x})$ has critical points at $(\nabla_{\boldsymbol{x}} f)(\boldsymbol{x}) = 0$

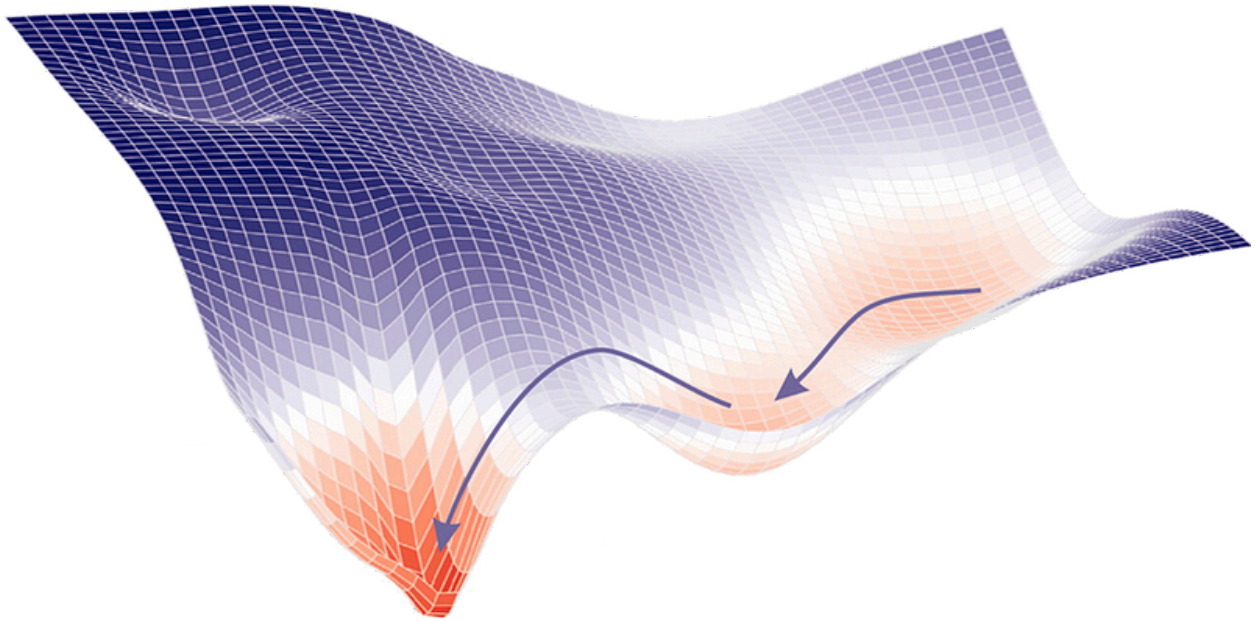


With optimization, we attempt to find minima of loss functions

Review

Gradients are important in deep learning for two reasons:

Reason 2: For problems without analytical solutions, the gradient (slope) is necessary for gradient descent



Review

First, we derived the solution to linear regression

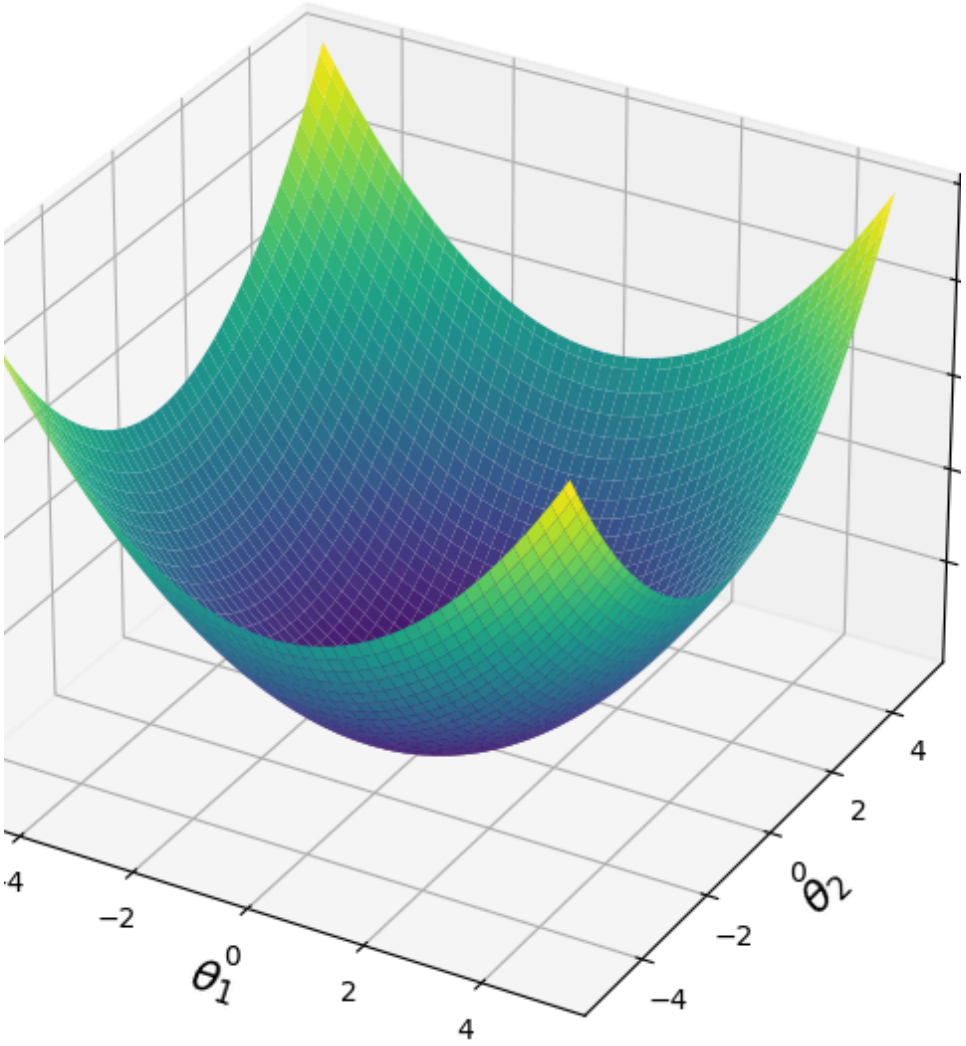
$$\mathcal{L}(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{\theta}) = \sum_{i=1}^{n} \left(f\left(\boldsymbol{x}_{[i]}, \boldsymbol{\theta}\right) - \boldsymbol{y}_{[i]} \right)^2$$

$$\mathcal{L}(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{\theta}) = \left(\boldsymbol{Y} - \overline{\boldsymbol{X}}\boldsymbol{\theta}\right)^\top \left(\boldsymbol{Y} - \overline{\boldsymbol{X}}\boldsymbol{\theta}\right)$$

$$\mathcal{L}(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{\theta}) = \underbrace{\underbrace{\left(\boldsymbol{Y} - \overline{\boldsymbol{X}}\boldsymbol{\theta}\right)^\top}_{\text{Linear function of } \theta} \underbrace{\left(\boldsymbol{Y} - \overline{\boldsymbol{X}}\boldsymbol{\theta}\right)}_{\text{Linear function of } \theta}}_{\text{Quadratic function of } \theta}$$

Review

A quadratic function has a single critical point, which must be a global minimum



Review

We found the analytical solution for linear regression by finding where the gradient was zero and solving for $\boldsymbol{\theta}$

$$(\nabla_{\boldsymbol{\theta}} \mathcal{L})(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{\theta}) = 0$$

$$\boldsymbol{\theta} = \left(\overline{\boldsymbol{X}}^\top \overline{\boldsymbol{X}}\right)^{-1} \overline{\boldsymbol{X}}^\top \boldsymbol{Y}$$

Which solves

$$\arg\min_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{\theta})$$

Review

For neural networks, the square error loss is no longer quadratic

$$\mathcal{L}(x, y, \boldsymbol{\theta}) = (f(x, \boldsymbol{\theta}) - y)^2 \qquad \text{Loss function}$$

$$f(x, \boldsymbol{\theta}) = \sigma(\theta_0 + \theta_1 x) \qquad \text{Neural network model}$$

Now, we plug the model f into the loss function

$$\mathcal{L}(x, y, \boldsymbol{\theta}) = (\sigma(\theta_0 + \theta_1 x) - y)^2$$

$$\mathcal{L}(x, y, \boldsymbol{\theta}) = \underbrace{(\sigma(\theta_0 + \theta_1 x) - y)}_{\text{Nonlinear function of } \theta}\underbrace{(\sigma(\theta_0 + \theta_1 x) - y)}_{\text{Nonlinear function of } \theta}$$

There is no analytical solution for $\boldsymbol{\theta}$

Review

Instead, we found neural network parameters through gradient descent

Gradient descent is an optimization method for differentiable functions

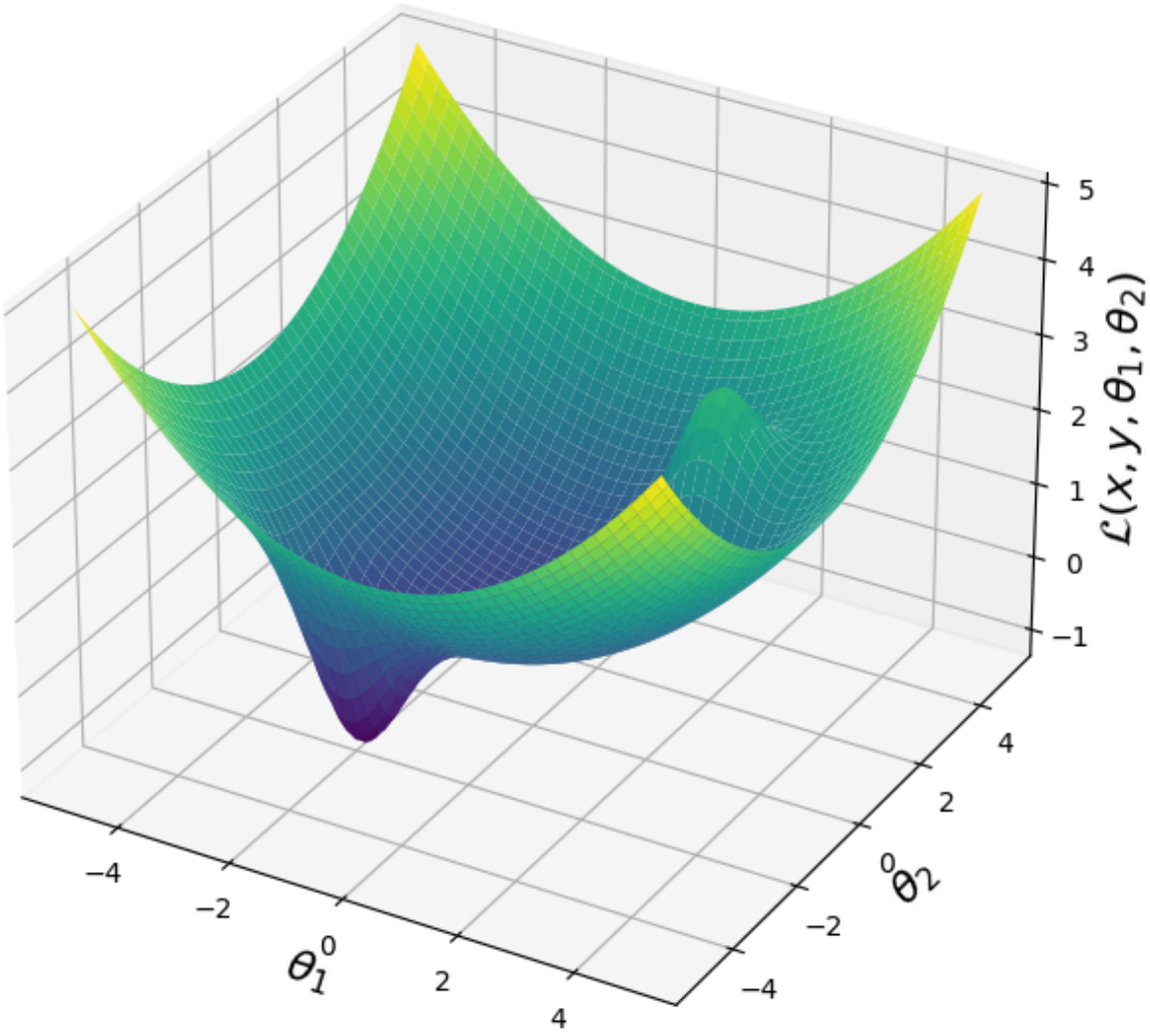
We went over both the intuition and mathematical definitions

Review





Review


5
4
3
2
1
0
−1
$\mathcal{L}(x, y, \theta_1, \theta_2)$
−4
−2
θ_1^0
2
4
4
2
θ_2^0
−2
−4

Review

The gradient descent algorithm:

procedure GRADIENT DESCENT($\boldsymbol{X}$, $\boldsymbol{Y}$, $\mathcal{L}$, t, α)
  for $i \in \{1...t\}$ **do**
    ▷ Compute the gradient of the loss
    $\boldsymbol{J} \leftarrow (\nabla_{\boldsymbol{\theta}} \mathcal{L})(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{\theta})$
    ▷ Update the parameters using the negative gradient
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \cdot \boldsymbol{J}$
  end
  return $\boldsymbol{\theta}$
end

Review

We derived the $\nabla_{\boldsymbol{\theta}} \mathcal{L}$ for deep neural networks using the chain rule

$$(\nabla_{\boldsymbol{\theta}} \mathcal{L})(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{\theta}) = \sum_{i=1}^{n} 2\Big(f\Big(\boldsymbol{x}_{[i]}, \boldsymbol{\theta}\Big) - \boldsymbol{y}_{[i]}\Big)(\nabla_{\boldsymbol{\theta}} f)\Big(\boldsymbol{x}_{[i]}, \boldsymbol{\theta}\Big)$$

$$(\nabla_{\boldsymbol{\theta}} f)(\boldsymbol{x}, \boldsymbol{\theta}) = (\nabla_{\boldsymbol{\varphi}, \boldsymbol{\psi}, \ldots, \boldsymbol{\xi}} f)\left(\boldsymbol{x}, \begin{bmatrix} \boldsymbol{\varphi} \\ \boldsymbol{\psi} \\ \vdots \\ \boldsymbol{\xi} \end{bmatrix}\right) = \begin{bmatrix} (\nabla_{\boldsymbol{\varphi}} f_1)(\boldsymbol{x}, \boldsymbol{\varphi}) \\ (\nabla_{\boldsymbol{\psi}} f_2)(\boldsymbol{z}_1, \boldsymbol{\psi}) \\ \vdots \\ (\nabla_{\boldsymbol{\xi}} f_\ell)(\boldsymbol{z}_{\ell-1}, \boldsymbol{\xi}) \end{bmatrix}$$

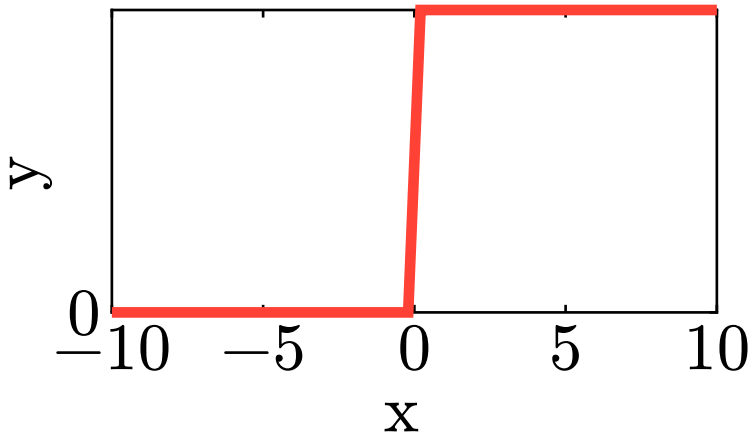
$$(\nabla_{\boldsymbol{\xi}} f_\ell)(\boldsymbol{z}_{\ell-1}, \boldsymbol{\xi}) = (\sigma(\boldsymbol{\xi}^\top \overline{\boldsymbol{z}}_{\ell-1}) \odot (1 - \sigma(\boldsymbol{\xi}^\top \overline{\boldsymbol{z}}_{\ell-1})))\overline{\boldsymbol{z}}_{\ell-1}$$

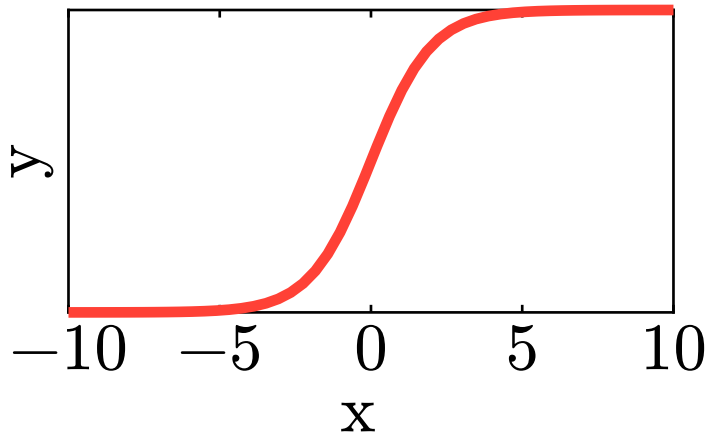
Review

The heaviside step function was not differentiable

We could not use it with gradient descent

Replaced heaviside with smooth approximation (sigmoid)





Review

In `torch`, we backpropagate through a graph of operations

```
import torch
optimizer = torch.optim.SGD(params, lr=0.0001)

def L(model, X, Y):
  ...
# Pytorch will record a graph of all operations
# Everytime you do theta @ x, it stores inputs and outputs
loss = L(X, Y, model) # compute loss
# Traverse the graph backward and compute the gradient
loss.backward() # Sets .grad attribute on each parameter
optimizer.step() # Update the parameters using .grad
optimizer.zero_grad() # Set .grad to zero, DO NOT FORGET!!
```

Review

First, a video of one application of gradient descent

https://youtu.be/kGDO2e_qiyI?si=ZopZKy-6WQ4B0csX

Time for some interactive torch coding

https://colab.research.google.com/drive/1EXhGq5mcKcYkKOdbsXjUoJaSKs-J78yk?usp=sharing

Classification

Classification

Many problems in ML can be reduced to **regression** or **classification**

Regression asks how many

- How long will I live?
- How much rain will there be tomorrow?
- How far away is this object?

Classification asks which one

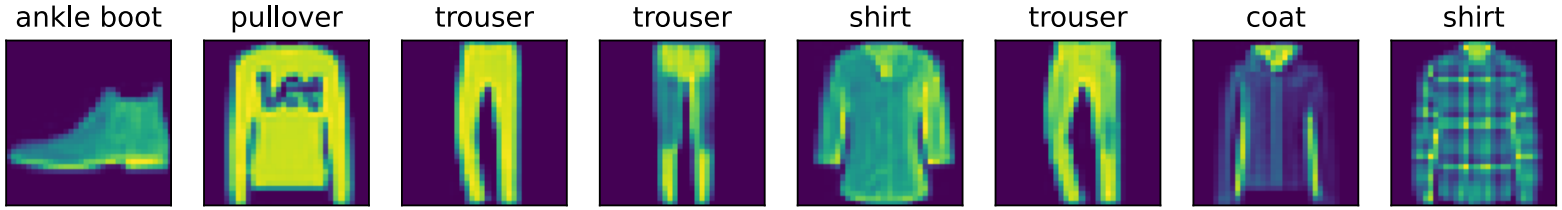
- Is this a dog or muffin?
- Will it rain tomorrow? Yes or no?
- What color is this object?

So far, we only looked at regression. Now, let us look at classification

Classification

Task: Given a picture of clothes, predict the text description

$X : \mathbb{Z}_{0,255}^{32\times32}$



Y : {T-shirt, Trouser, Pullover, Dress, Coat,

Sandal, Shirt, Sneaker, Bag, Ankle boot}

Approach: Learn $\boldsymbol{\theta}$ that produce **conditional probabilities**

$$f(\boldsymbol{x}, \boldsymbol{\theta}) = P(\boldsymbol{y} \mid \boldsymbol{x}) = P\left(\begin{bmatrix} \text{T-Shirt} \\ \text{Trouser} \\ \vdots \end{bmatrix} \;\middle|\; \right) = \begin{bmatrix} 0.2 \\ 0.01 \\ \vdots \end{bmatrix}$$

Probability Theory

Probability Theory

Classification tasks require **probability theory**, so let us review

In probability, we have **experiments** and **outcomes**

An experiment yields one of many possible outcomes

Experiment	Outcome
Flip a coin	Heads
Walk outside	Rain
Grab clothing from closest	Coat

Probability Theory

The **sample space** S defines all possible outcomes for an experiment

Experiment	Sample Space S
Flip a coin	$S = \{\text{heads}, \text{tails}\}$
Walk outside	$S = \{\text{rain}, \text{sun}, \text{wind}, \text{cloud}\}$
Take clothing from closet	$S = \{\text{T-shirt}, \text{Trouser}, \text{Pullover}, \text{Dress}, \text{Coat}, \text{Sandal}, \text{Shirt}, \text{Sneaker}, \text{Bag}, \text{Ankle boot}\}$

Probability Theory

An **event** E is a specific subset of the sample space

Experiment	Sample Space	Event
Flip a coin	$S = \{\text{heads}, \text{tails}\}$	$E = \{\text{heads}\}$
Walk outside	$S = \{\text{rain}, \text{sun}, \text{wind}, \text{cloud}\}$	$E = \{\text{rain}, \text{wind}\}$
Take from closet	$\{\text{T-shirt}, \text{Trouser}, \text{Pullover}, \text{Dress}, \text{Coat}, \text{Sandal}, \text{Shirt}, \text{Sneaker}, \text{Bag}, \text{Ankle boot}\}$	$E = \{\text{Shirt}, \text{T-Shirt}, \text{Coat}\}$

Probability Theory

The **probability** measures how likely an event is to occur

The probability must be between 0 (never occurs) and 1 (always occurs)

$$0 \leq P(E) \leq 1$$

Experiment	Probabilities
Flip a coin	$P(\text{heads}) = 0.5$
Walk outside	$P(\text{rain}) = 0.15$
Take from closet	$P(\text{Shirt}) = 0.1$

Probability Theory

When we define P as a function over the sample space S, we call it a **Probability Distribution**

For a discrete sample space S, the distribution is a **Probability Mass Function (PMF)** that maps each outcome to a probability

$$P : S \to [0, 1]$$

The distribution must sum to one over the sample space

$$\sum_{x \in S} P(x) = 1$$

Probability Theory

$$\sum_{x \in S} P(x) = 1$$

Flip a coin

$$\{P(\text{heads}) = 0.5, P(\text{tails}) = 0.5\}$$

Take clothing from closet

$$\{P(\text{T-shirt}) = 0.1, P(\text{Trouser}) = 0.08, P(\text{Pullover}) = 0.12, ...\}$$

Probability Theory

We can also define P over the event space E

$$P : E \rightarrow [0, 1]$$

Question: Does it still sum to one?

Answer: Not always!

Flip a coin

$$P(\{\text{heads}\}) = 0.5$$

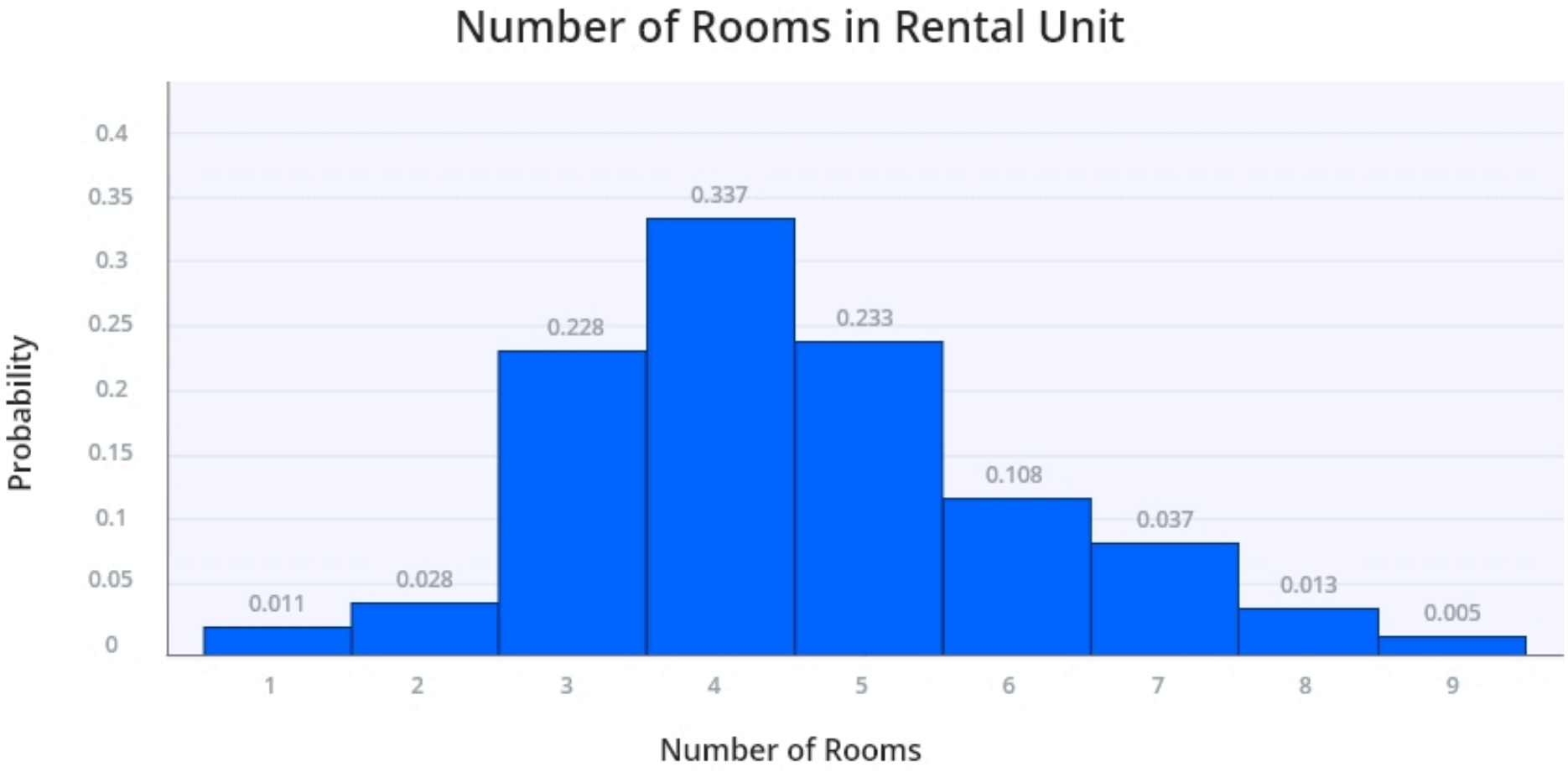
$$P(\{\}) = 0$$

$$P(\{\text{heads}, \text{tails}\}) = 1.0$$

Probability Theory

P is a function, so we can plot it

Probability Theory



Probability Theory

Events can overlap with each other

- Disjoint events
- Conditionally dependent events

Probability Theory

Two events A, B are **disjoint** if A and B cannot occur together

$$P(A, B) = 0$$

Flip a coin

$$P(\text{Heads}) = 0.5, P(\text{Tails}) = 0.5$$
$$P(\text{Heads}, \text{Tails}) = 0$$

Be careful!

Walk outside

$$P(\text{Rain}) = 0.1, P(\text{Cloud}) = 0.2$$
$$P(\text{Rain} \cap \text{Cloud}) \neq 0$$

Probability Theory

If B provides information about A, they are **conditionally dependent**

$$P(A \mid B) \neq P(A)$$

$$P(\text{cloud}) = 0.2, P(\text{rain}) = 0.1$$

$$P(\text{rain} \mid \text{cloud}) = 0.5$$

$$P(A \mid B) = \frac{P(A, B)}{P(B)}$$

$$P(\text{Rain}, \text{Cloud}) = 0.1$$

Walk outside

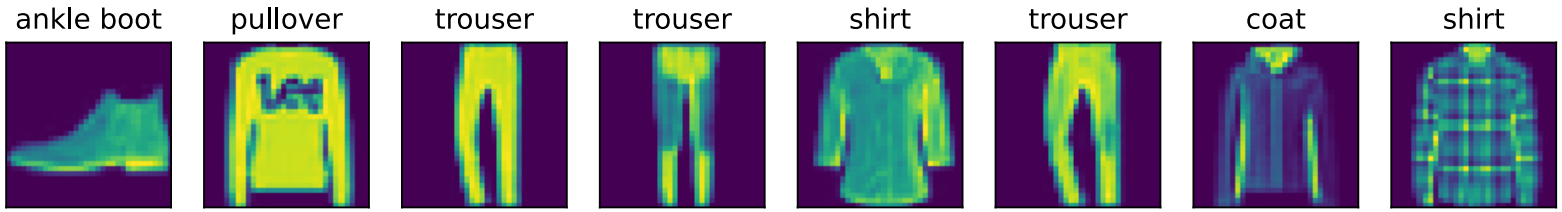
$$P(\text{Cloud}) = 0.2$$

$$P(\text{Rain} \mid \text{Cloud}) = \frac{0.1}{0.2} = 0.5$$

Probability Theory

Task: Given a picture of clothes, predict the text description

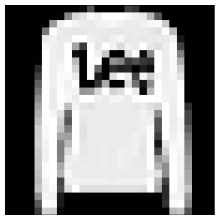
$X : \mathbb{Z}_{0,255}^{32\times32}$



Y : {T-shirt, Trouser, Pullover, Dress, Coat,

Sandal, Shirt, Sneaker, Bag, Ankle boot}

Approach: Learn $\boldsymbol{\theta}$ that produce **conditional probabilities**

$$f(\boldsymbol{x}, \boldsymbol{\theta}) = P(\boldsymbol{y} \mid \boldsymbol{x}) = P\left(\begin{bmatrix} \text{T-Shirt} \\ \text{Trouser} \\ \vdots \end{bmatrix} \;\middle|\; \text{  } \right) = \begin{bmatrix} 0.2 \\ 0.01 \\ \vdots \end{bmatrix}$$

Classification Model

Classification Model

We will start with a multivariate linear model

$$f(\boldsymbol{x}, \boldsymbol{\theta}) = \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}$$

We want our model to predict the probability of each outcome

Question: What is the function signature of f?

Answer: $f : \mathbb{R}^{d_x} \times \Theta \mapsto \mathbb{R}^{d_y}$

Question: Can we use this model to predict probabilities?

Answer: No! Outcome probabilities must be $\in [0, 1]$ and sum to one

Classification Model

How can we model a distribution over outcomes?

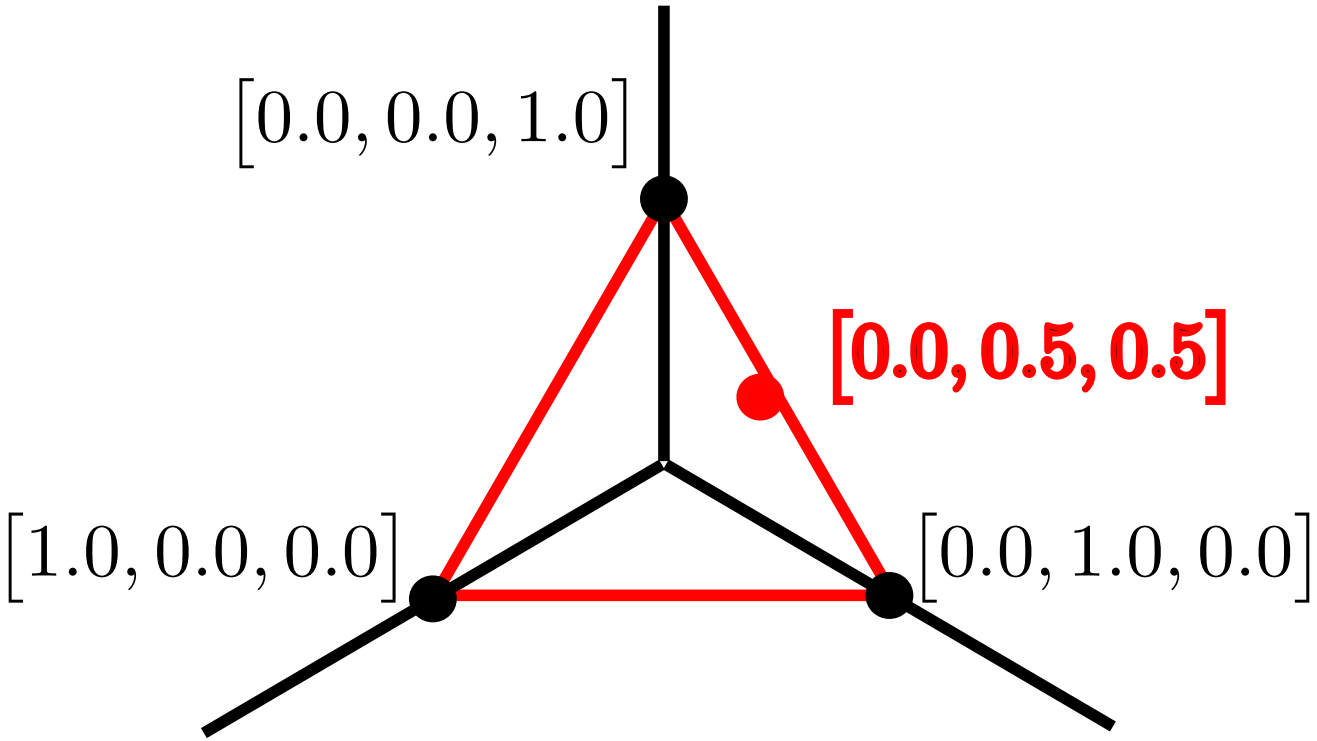
$$\boldsymbol{v} = \left\{ \begin{bmatrix} v_1 \\ \vdots \\ v_{d_y} \end{bmatrix} \;\middle|\; \sum_{i=1}^{d_y} v_i = 1; \quad v_i \in [0, 1] \right\}$$

There is special notation for this vector, called the **simplex**

$$\Delta^{d_y - 1}$$

Classification Model

The simplex Δ^{k-1} is an $k-1$-dimensional triangle in k-dimensional space



Classification Model

It has only $k-1$ free variables, because $x_k = 1 - \sum_{i=1}^{k-1} x_i$

Classification Model

$$f : \mathbb{R}^{d_x} \times \Theta \mapsto \mathbb{R}^{d_y}$$

So we need a function that maps to the simplex

$$g : \mathbb{R}^{d_y} \mapsto \Delta^{d_y - 1}$$

Then, we can combine f and g

$$g(f) : \mathbb{R}^{d_x} \times \Theta \mapsto \Delta^{d_y - 1}$$

Classification Model

We need a function g

$$g : \mathbb{R}^{d_y} \mapsto \Delta^{d_y - 1}$$

There are many functions that can do this

One example is dividing by the L_1 norm:

$$g(\boldsymbol{x}) = \frac{\boldsymbol{x}}{\sum_{i=1}^{d_y} x_i}$$

In deep learning we often use the **softmax** function. When combined with the classification loss the gradient is linear, making learning faster

Classification Model

The softmax function maps real numbers to the simplex (probabilities)

$$\text{softmax} : \mathbb{R}^k \mapsto \Delta^{k-1}$$

$$\text{softmax}\left(\begin{bmatrix} x_1 \\ \vdots \\ x_k \end{bmatrix}\right) = \frac{e^{\boldsymbol{x}}}{\sum_{i=1}^{k} e^{x_i}} = \begin{bmatrix} \frac{e^{x_1}}{e^{x_1}+e^{x_2}+\ldots e^{x_k}} \\ \frac{e^{x_2}}{e^{x_1}+e^{x_2}+\ldots e^{x_k}} \\ \vdots \\ \frac{e^{x_k}}{e^{x_1}+e^{x_2}+\ldots e^{x_k}} \end{bmatrix}$$

If we attach it to our linear model, we can output probabilities!

$$f(\boldsymbol{x}, \boldsymbol{\theta}) = \text{softmax}(\boldsymbol{\theta}^\top \overline{\boldsymbol{x}})$$

Classification Model

Naturally, we can use the same method with a deep neural network

$$f_1(\boldsymbol{x}, \boldsymbol{\varphi}) = \sigma(\boldsymbol{\varphi}^\top \overline{\boldsymbol{x}})$$
$$\vdots$$
$$f_\ell(\boldsymbol{x}, \boldsymbol{\xi}) = \text{softmax}(\boldsymbol{\xi}^\top \overline{\boldsymbol{x}})$$

Now, our neural network can output probabilities

$$f(\boldsymbol{x}, \boldsymbol{\theta}) = \begin{bmatrix} P\left(\text{Shirt} \mid \text{  }\right) \\ P\left(\text{Bag} \mid \text{  }\right) \\ \vdots \end{bmatrix} = \begin{bmatrix} 0.25 \\ 0.08 \\ \vdots \end{bmatrix}$$

Classification Model

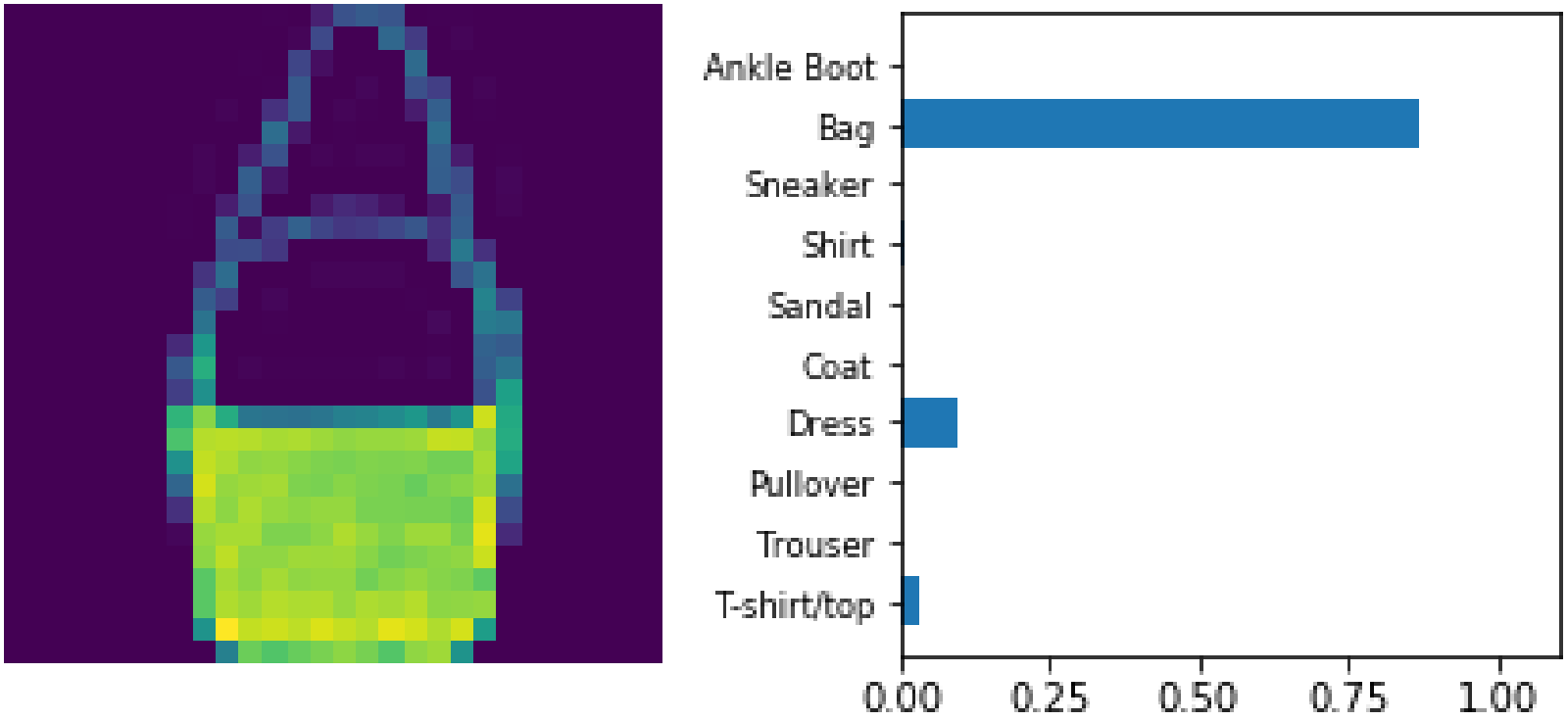
Question: Why do we output probabilities instead of a binary values

$$f(\boldsymbol{x}, \boldsymbol{\theta}) = \begin{bmatrix} P\Big(\text{Shirt} \mid \text{  }\Big) \\ P\Big(\text{Bag} \mid \text{  }\Big) \\ \vdots \end{bmatrix} = \begin{bmatrix} 0.25 \\ 0.08 \\ \vdots \end{bmatrix}; \quad f(\boldsymbol{x}, \boldsymbol{\theta}) = \begin{bmatrix} 1 \\ 0 \\ \vdots \end{bmatrix}$$

Answer 1: Outputting probabilities results in differentiable functions

Answer 2: We report uncertainty, which is useful in many applications

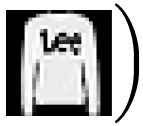
Classification Model



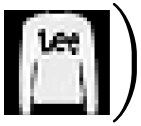
Classification Loss

Classification Loss

We consider the label $\boldsymbol{y}_{[i]}$ as a conditional distribution

$$P\left(\boldsymbol{y}_{[i]} \mid \boldsymbol{x}_{[i]}\right) = \begin{bmatrix} P\left(\text{Shirt} \mid \text{  }\right) \\ P\left(\text{Bag} \mid \text{  }\right) \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Our neural network also outputs some distribution

$$f\left(\boldsymbol{x}_{[i]}, \boldsymbol{\theta}\right) = \begin{bmatrix} P\left(\text{Shirt} \mid \text{  }\right) \\ P\left(\text{Bag} \mid \text{  }\right) \end{bmatrix} = \begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix}$$

What loss function should we use for classification?

Classification Loss

$$f\left(\boldsymbol{x}_{[i]}, \boldsymbol{\theta}\right) = \begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix}; \quad \boldsymbol{y}_{[i]} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

We could use the square error like linear regression

$$(0.6 - 1)^2 + (0.4 - 0)^2$$

This can work, but in reality it does not work well

Distributions have special properties (positive, sum to one) that square error does not consider

Instead, we use the **cross-entropy loss**

Let us derive it

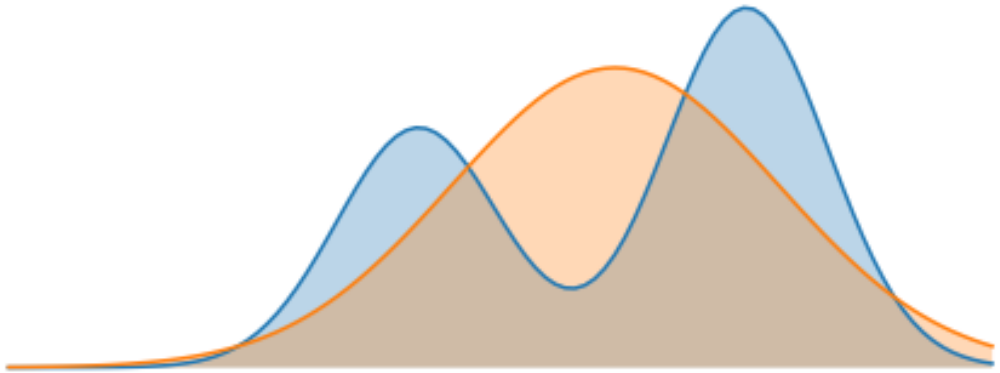
Classification Loss

We can model $f(\boldsymbol{x}, \boldsymbol{\theta})$ and $\boldsymbol{y}$ as probability distributions

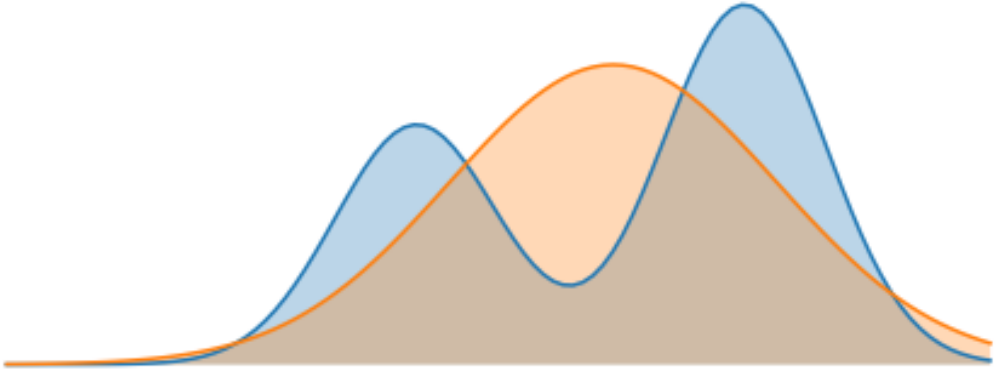
Loss should measure the error between distributions $f(\boldsymbol{x}, \boldsymbol{\theta})$ and $\boldsymbol{y}$

How do we measure the error between probability distributions?

We use the **Kullback-Leibler Divergence (KL)**



Classification Loss



$$\mathrm{KL}(P, Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$$

Classification Loss

First, write down KL-divergence

$$\mathrm{KL}(P, Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$$

Plug in our two distributions $P = \boldsymbol{y}$ and $Q = f$

$$\mathrm{KL}(P(\boldsymbol{y} \mid \boldsymbol{x}), f(\boldsymbol{x}, \boldsymbol{\theta})) = \sum_{i=1}^{d_y} P(y_i \mid \boldsymbol{x}) \log \frac{P(y_i \mid \boldsymbol{x})}{f(\boldsymbol{x}, \boldsymbol{\theta})_i}$$

Classification Loss

$$\mathrm{KL}(P(\boldsymbol{y} \mid \boldsymbol{x}), f(\boldsymbol{x}, \boldsymbol{\theta})) = \sum_{i=1}^{d_y} P(y_i \mid \boldsymbol{x}) \log \frac{P(y_i \mid \boldsymbol{x})}{f(\boldsymbol{x}, \boldsymbol{\theta})_i}$$

Rewrite the logarithm using the sum rule of logarithms

$$\mathrm{KL}(P(\boldsymbol{y} \mid \boldsymbol{x}), f(\boldsymbol{x}, \boldsymbol{\theta})) = \sum_{i=1}^{d_y} P(y_i \mid \boldsymbol{x})(\log P(y_i \mid \boldsymbol{x}) - \log f(\boldsymbol{x}, \boldsymbol{\theta})_i)$$

Classification Loss

$$\mathrm{KL}(P(\boldsymbol{y} \mid \boldsymbol{x}), f(\boldsymbol{x}, \boldsymbol{\theta})) = \sum_{i=1}^{d_y} P(y_i \mid \boldsymbol{x})(\log P(y_i \mid \boldsymbol{x}) - \log f(\boldsymbol{x}, \boldsymbol{\theta})_i)$$

Split the sum into two parts

$$= \sum_{i=1}^{d_y} P(y_i \mid \boldsymbol{x}) \log P(y_i \mid \boldsymbol{x}) - \sum_{i=1}^{d_y} P(y_i \mid \boldsymbol{x}) \log f(\boldsymbol{x}, \boldsymbol{\theta})_i$$

Classification Loss

$$= \sum_{i=1}^{d_y} P(y_i \mid \boldsymbol{x}) \log P(y_i \mid \boldsymbol{x}) - \sum_{i=1}^{d_y} P(y_i \mid \boldsymbol{x}) \log f(\boldsymbol{x}, \boldsymbol{\theta})_i$$

The first term is constant, and we will minimize the loss. So $\arg\min_{\boldsymbol{\theta}} \mathcal{L} + k = \arg\min_{\boldsymbol{\theta}} \mathcal{L}$. Therefore, we can ignore the first term.

$$= -\sum_{i=1}^{d_y} P(y_i \mid \boldsymbol{x}) \log f(\boldsymbol{x}, \boldsymbol{\theta})_i$$

This is the loss for a classification task! We call this the **cross-entropy** loss function

Classification Loss

Example:

$$\mathcal{L}(\boldsymbol{x}, \boldsymbol{y}, \boldsymbol{\theta}) = -\sum_{i=1}^{d_y} P(y_i \mid \boldsymbol{x}) \log f(\boldsymbol{x}, \boldsymbol{\theta})_i; \quad f(\boldsymbol{x}, \boldsymbol{\theta}) = \begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix}; \quad \boldsymbol{y} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

$$= -[P(y_1 \mid \boldsymbol{x}) \log f(\boldsymbol{x}, \boldsymbol{\theta})_1 + P(y_2 \mid \boldsymbol{x}) \log f(\boldsymbol{x}, \boldsymbol{\theta})_2]$$

$$= -[1 \cdot \log 0.6 + 0 \cdot \log 0.4]$$

$$= -[-0.22]$$

$$= 0.22$$

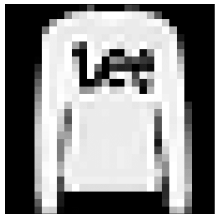
Classification Loss

$$\mathcal{L}(\boldsymbol{x}, \boldsymbol{y}, \boldsymbol{\theta}) = -\sum_{i=1}^{d_y} P(y_i \mid \boldsymbol{x}) \log f(\boldsymbol{x}, \boldsymbol{\theta})_i$$

By minimizing the loss, we make $f(\boldsymbol{x}, \boldsymbol{\theta}) = P(\boldsymbol{y} \mid \boldsymbol{x})$

$$\arg\min_{\theta} \mathcal{L}(\boldsymbol{x}, \boldsymbol{y}, \boldsymbol{\theta}) = \arg\min_{\theta} \left[-\sum_{i=1}^{d_y} P(y_i \mid \boldsymbol{x}) \log f(\boldsymbol{x}, \boldsymbol{\theta})_i \right]$$

$$f(\boldsymbol{x}, \boldsymbol{\theta}) = P(\boldsymbol{y} \mid \boldsymbol{x}) = P\left(\begin{bmatrix} \text{boot} \\ \text{dress} \\ \vdots \end{bmatrix} \,\middle|\, \right)$$



Classification Loss

Our loss was just for a single image

$$\mathcal{L}(\boldsymbol{x}, \boldsymbol{y}, \boldsymbol{\theta}) = \left[-\sum_{i=1}^{d_y} P(y_i \mid \boldsymbol{x}) \log f(\boldsymbol{x}, \boldsymbol{\theta})_i \right]$$

Find $\boldsymbol{\theta}$ that minimize the loss over the whole dataset

$$\mathcal{L}(\boldsymbol{x}, \boldsymbol{y}, \boldsymbol{\theta}) = \left[-\sum_{j=1}^{n} \sum_{i=1}^{d_y} P\left(y_{[j],i} \mid \boldsymbol{x}_{[j]}\right) \log f\left(\boldsymbol{x}_{[j]}, \boldsymbol{\theta}\right)_i \right]$$

Classification Optimization

Classification Optimization

Find $\boldsymbol{\theta}$ just like before, using gradient descent

The gradients are the same as before except the last layer

I will not derive any more gradients, but the softmax gradient nearly identical to the sigmoid function gradient

$$(\nabla_{\boldsymbol{\theta}} \operatorname{softmax})(\boldsymbol{z}) = \operatorname{softmax}(\boldsymbol{z}) \odot (1 - \operatorname{softmax}(\boldsymbol{z}))$$

Softmax is a multi-class generalization of the sigmoid function

Coding

Coding

You have everything you need to solve deep learning tasks!

1. Regression
2. Classification

Every interesting task (chatbot, self driving car, etc):

1. Construct a deep neural network
2. Compute regression or classification loss
3. Optimize with gradient descent

The rest of this course will examine neural network architectures

Coding

https://colab.research.google.com/drive/16x5yGf8wVhEGY48r0Wgi8uwlsS0AGVC-?usp=sharing

Coding

Homework 2 is released, you have two weeks to complete it

https://colab.research.google.com/drive/1etlW__tthA49DRNVHnTU_nE9yqpEz01l?usp=sharing